Parallelizing a Two-Dimensional Jacobi Iteration with MPI and OpenMP

DSA5208 Project 3

Fan Weiqi Student ID: A0354521L
Liu Peihan Student ID: A0357850W
Chen Yiru Student ID: A0327764U

GitHub repository: `https://github.com/yiruchenn/DSA5208`

ABSTRACT

We implement a fixed-iteration, five-point Jacobi method on a two-dimensional grid in serial C++ and in hybrid MPI+OpenMP C++. MPI distributes contiguous row blocks; OpenMP parallelizes updates within a block. We compare blocking halo exchange with nonblocking exchange that permits computation of interior rows while messages are outstanding. A discrete-eigenmode reference solution and cell-by-cell serial comparisons assess correctness. The supplied Vanda-labelled benchmark records show numerical errors below 1.9×10^{-14}, but poor parallel scaling: for a 1024^2 grid and 400 iterations, the serial median is 0.2525 s, while two MPI ranks with one thread each require about 10.41 s. At four ranks, nonblocking exchange is faster than blocking exchange in this dataset, yet both remain much slower than serial. We report these observations without attributing the abrupt slowdown to an unmeasured cause.

Keywords: Jacobi iteration; MPI; OpenMP; halo exchange; strong scaling; weak scaling.

I INTRODUCTION

Jacobi iteration updates each unknown from its neighbours in the previous iteration [1]. Its regular stencil admits substantial work within a process, but a distributed implementation cannot advance without values from adjacent row blocks. This makes it a useful case for studying the interaction of local parallelism, communication, and synchronization. Our questions are whether the hybrid implementation preserves the serial numerical result, how MPI rank count and OpenMP thread count affect strong and weak scaling, and whether overlapping halo communication improves the measured runtime.

The program performs a prescribed number of Jacobi steps with fixed zero boundary values. It can be interpreted as discrete relaxation or diffusion-like smoothing. It does not implement a physical time integrator or stop at a convergence tolerance. No external dataset is used, so the assignment's conditional requirement for conclusions about real data does not apply.

Our comparison has three levels. First, a separate serial executable establishes the one-core reference and clarifies the numerical recurrence. Second, the hybrid executable provides a single-rank threaded case, allowing OpenMP effects to be observed without an inter-rank boundary. Third, multi-rank runs introduce MPI halo exchange. Within each parallel configuration, `blocking` and `overlap` modes change the communication schedule without changing the Jacobi formula. This structure lets us distinguish observations associated with threads, ranks, and communication mode, while recognizing that the available measurements cannot identify every source of overhead.

The principal contribution of this report is therefore an evidence-based evaluation of the supplied implementation, including an unfavorable result. Parallel programming is not validated solely by showing agreement with a serial answer: the runtime measurements must also be interpreted with their exact baseline, problem size, and timing scope. In this dataset, correctness and performance lead to different conclusions. We first establish the recurrence and the independent numerical reference, then describe the parallel design, validate its output, and finally examine the Vanda measurements and their limitations.

II ALGORITHM AND NUMERICAL REFERENCE

For interior indices $1 \le i \le n_x$, $1 \le j \le n_y$, the update is

$$u_{i,j}^{(k+1)} = \tfrac{1}{4}\left(u_{i-1,j}^{(k)} + u_{i+1,j}^{(k)} + u_{i,j-1}^{(k)} + u_{i,j+1}^{(k)}\right), \tag{1}$$

with $u = 0$ on all four physical boundaries. The serial and parallel programs initialize the same two-mode field,

$$u_{i,j}^{(0)} = \sin\frac{\pi i}{n_x+1}\sin\frac{\pi j}{n_y+1} + \tfrac{1}{4}\sin\frac{2\pi i}{n_x+1}\sin\frac{3\pi j}{n_y+1}. \tag{2}$$

For this *discrete* update, the sine mode (p, q) is an eigenvector with amplification factor

$$\lambda_{p,q} = \tfrac{1}{2}\left[\cos\frac{p\pi}{n_x+1} + \cos\frac{q\pi}{n_y+1}\right]. \tag{3}$$

Consequently, an independent reference after K steps is

$$u_{i,j}^{(K)} = \lambda_{1,1}^{K}\sin\frac{\pi i}{n_x+1}\sin\frac{\pi j}{n_y+1} \tag{4}$$

$$+ \tfrac{1}{4}\lambda_{2,3}^{K}\sin\frac{2\pi i}{n_x+1}\sin\frac{3\pi j}{n_y+1}. \tag{5}$$

Equation (5) is exact for the mathematical Jacobi recurrence up to floating-point roundoff; it is not a claimed exact solution of a continuous heat equation. Each implementation keeps separate `current` and `next` arrays, so all updates in a step read the same old field.

The eigenvalue formula follows directly from the sine addition identity. For a sine factor in the row direction, the sum of its values one row above and below equals $2\cos(p\pi/(n_x+1))$ times its value at the current row. The analogous column sum contributes $2\cos(q\pi/(n_y+1))$. Dividing the four-neighbour sum by four gives $\lambda_{p,q}$. Because the recurrence is linear, the two modes in Equation (2) evolve independently, giving Equation (5). This reference is particularly useful here: it tests an entire K-step field without assuming that the serial program itself is correct.

With zero boundary values and no forcing term, the fixed point of this discrete Laplace relaxation is the zero field. The chosen nonzero initial condition therefore supplies a meaningful finite-step state for testing the update, rather than a nontrivial steady-state solution. The two-mode construction also makes it possible to verify all reported benchmark configurations against Equation (5) at the exact iteration count used by the code.

One step performs one constant-cost stencil calculation for each of the $n_x n_y$ interior cells. For fixed K, the serial work is $\Theta(Kn_xn_y)$ and storage is $\Theta(n_xn_y)$ for each of the two buffers. Boundary values remain zero and are never updated. There is no global convergence reduction inside the iteration loop; synchronization arises from the dependence on neighbouring rank-owned rows. Thus, unlike an independent map over cells, a distributed step cannot simply proceed without communication or an equivalent mechanism for supplying the preceding step's boundary values.

III PARALLEL DESIGN

A DOMAIN DECOMPOSITION AND THREADING

With R MPI ranks, rank r owns a contiguous strip of approximately n_x/R complete interior rows; the first $n_x \bmod R$ ranks receive one additional row. Every rank stores one ghost row above and below its owned rows. At each step it exchanges its first and last owned rows with neighbouring ranks; physical top and bottom boundaries remain zero. As each rank owns complete rows, no left/right MPI exchange is needed. MPI calls occur outside OpenMP regions under `MPI_THREAD_FUNNELED`. OpenMP uses a parallel row loop with runtime scheduling; the reported performance experiments use `static` scheduling only.

More precisely, rank r receives $\lfloor n_x/R \rfloor + 1$ rows if $r < n_x \bmod R$, and $\lfloor n_x/R \rfloor$ otherwise. The first global row owned by rank r is $1 + r\lfloor n_x/R \rfloor + \min(r, n_x \bmod R)$. This gives a contiguous, nonoverlapping partition and keeps rank workloads within one row of one another. The program rejects $R > n_x$, avoiding empty

subdomains. On the measured 1024-row strong-scaling grid, both $R = 2$ and $R = 4$ divide the rows exactly, so row-count imbalance is not a natural explanation for the observed large timing gap.

The decomposition deliberately exchanges only the n_y interior values of each boundary row. A rank at a physical top or bottom edge has one MPI neighbour; an interior rank has two. In the measured $n_y = 1024$ case, one row contains $1024 \times 8 = 8192$ bytes in double precision. At two ranks, each rank sends and receives one such row per iteration, or about 6.55 MB of combined send-plus-receive payload over 400 steps. At four ranks, an interior rank handles up to twice that payload. These are payload counts, not measured transfer times; MPI protocol costs, network topology, synchronization, and progress behaviour remain unknown.

B TWO HALO STRATEGIES

In `blocking` mode, two `MPI_Sendrecv` operations finish before any row is updated. In `overlap` mode, the rank posts nonblocking receives and sends, computes owned rows 2 through $m - 1$ (where m is its local row count), waits for all requests, then computes the first and last owned rows. This ordering is safe because the interior rows do not read incoming ghost rows. When a rank owns only one or two rows, little or no computation can overlap communication. The code uses double buffering in both modes and swaps arrays after every step. The point-to-point operations and completion requests follow the MPI interface [2]; citing that standard describes API semantics, not the version of MPI installed on Vanda, which is not recorded in the supplied files.

The two `MPI_Sendrecv` calls use complementary message tags: one moves the first owned row upward and receives from below; the other moves the last owned row downward and receives from above. In overlap mode, all receives are posted before sends. The old `current` field remains unchanged while communication is pending, while OpenMP writes only to `next`. Waiting before the boundary-row update ensures that every remote neighbour value belongs to the same old iteration. These ordering facts explain why overlap can change runtime without changing numerical semantics.

Overlap is an opportunity rather than a guarantee. A simplified blocking critical path is communication followed by computation. For overlap, the interior calculation and communication may share part of the critical path, but posting requests, waiting, and updating boundary rows still cost time. Nonblocking initiation does not by itself establish how much communication progresses during the interior calculation on the installed MPI implementation; the code's additional OpenMP row regions can also add overhead. Hence a speed difference must be measured rather than inferred merely from use of `MPI_Isend` and `MPI_Irecv`.

For fixed global dimensions and iteration count K, the ideal local stencil work decreases roughly as $O(Kn_xn_y/(RT))$, where T is the thread count per rank. An interior rank sends and receives up to two rows of n_y doubles per step. Thus the amount of local work falls with R while the halo length does not. Latency, bandwidth, thread overhead, and memory behaviour can therefore limit scaling. This model identifies plausible costs, not the measured cause of a particular slowdown.

C OPENMP SCHEDULING AND DESIGN TRADEOFFS

The OpenMP loop uses `schedule(runtime)` and the executable supports both static and dynamic schedules, as specified by the OpenMP API [3]. Because each row has the same number of columns and the same stencil operations, static scheduling is a reasonable default: it assigns rows without the per-chunk coordination required by dynamic scheduling. Dynamic scheduling can still be useful as a diagnostic option, but the available Vanda CSVs contain only static results. We therefore make no empirical claim about a static-versus-dynamic performance difference. The cited API specification is a programming reference, not evidence for the unrecorded OpenMP implementation version used in the benchmark.

At a fixed total core count, using more threads per rank reduces the number of MPI boundaries, while using more ranks may reduce each rank's memory footprint and change thread overhead. Neither effect has a guaranteed sign for runtime. The measured configurations permit some like-for-like comparisons, notably 1×2 against 2×1 at two cores and 2×2 against 4×1 at four cores. The comparison is still conditional on the MPI process placement and core binding, which the provided CSV does not document. A two-dimensional MPI partition could shorten halo boundaries at larger rank counts, but it is not implemented or measured here.

D COST MODEL AND EXPECTATIONS

A deliberately simple model clarifies what the scaling experiments test. Let a represent the cost per updated cell, b the transfer cost per halo value, ℓ a message-latency term, and $h(T)$ threading overhead. For an interior rank in blocking mode, a rough iteration-loop time is

$$t_{\text{block}}(R,T) \approx K\left[a\frac{n_x n_y}{RT} + bn_y + \ell + h(T)\right]. \tag{6}$$

The constants are illustrative and are not fitted to the CSV. For overlap, part of the communication term may coincide with the interior-row calculation; it cannot simply be deleted because the boundary rows wait for the halo. This model also omits cache effects, contention, MPI progress, and placement.

Under strong scaling, $n_x n_y$ is fixed, so the computation term shrinks as cores increase while the halo length stays at n_y. Even with an efficient runtime, the communication-to-computation ratio can worsen. Under the chosen weak-scaling rule, both dimensions grow approximately as $\sqrt{RT}$: work per core stays roughly constant but the 1D decomposition's halo row length grows approximately as $\sqrt{RT}$. Thus weak scaling need not be perfectly flat even in an otherwise healthy implementation. The model predicts a direction of pressure on efficiency, not the observed multi-second jump; interpreting that jump requires measurements of the terms the current CSV does not retain.

IV CORRECTNESS VALIDATION

The supplied `validate.py` compares each serial field with Equation (5), dumps complete serial and parallel fields, and checks every parallel cell against the serial reference with a 2×10^{-12} absolute tolerance. It also requires a maximum analytic error below 10^{-11} for its small regression cases. The supplied log records passes for five grid/step cases, including nonsquare grids, zero steps, nonuniform decompositions, one-row subdomains, one to four ranks, one or two threads per rank, both halo strategies, and a dynamic-scheduling check. The largest printed serial analytic error in that log is 4.72×10^{-15}; all printed parallel-versus-serial maximum cell differences are zero.

The Vanda-labelled strong and weak benchmark CSVs each record a maximum analytic error no larger than 1.821×10^{-14} across their configurations. These errors support numerical agreement, including in the runs with poor performance. The validation log itself does not identify a host or job ID, and the package's older local-status note describes tests against a simulated MPI API. Therefore we use the log as a supplied regression record, without claiming that every logged test was executed on Vanda.

Table 1 makes the small-grid evidence explicit. The zero-step case checks initialization and output without an update. The one-row-per-rank cases exercise the branch in which no interior row can be computed while halo exchange is outstanding. Nonsquare and nonuniform cases test indexing and the calculation of global row offsets. Longer runs exercise repeated buffer swaps and halo replacement. These cases target distinct failure modes; they are not merely repetitions of a square-grid example.

Table 1: Cases reported in the supplied validation log. Each serial maximum is relative to Equation (5); the logged parallel cell-wise differences against serial are zero.

n_x	n_y	Steps	Serial maximum error	Additional property
2	3	0	0	Initialization; one row per rank at $R = 2$
3	5	1	1.11×10^{-16}	One row per rank at $R = 3$
7	11	17	3.89×10^{-16}	Unequal row counts for some ranks
17	13	25	3.33×10^{-16}	Nonsquare grid; repeated exchanges
21	29	100	4.72×10^{-15}	Longer repeated iteration

For the full-field comparison, the maximum cell discrepancy is $\max_{i,j} |u_{i,j}^{\text{parallel}} - u_{i,j}^{\text{serial}}|$. The analytic maximum error is $\max_{i,j} |u_{i,j}^{(K)} - u_{i,j}^{\text{exact}}|$; the code also reports the ratio of the error and exact-field L_2 norms. The analytic comparison is stronger than a single checksum because many different fields can share a sum. The cell-wise serial comparison independently tests MPI gathering and local/global index mapping. Nevertheless, the serial and parallel programs share initialization and reference routines in `common.hpp`; the derived eigenmode identity is the mathematical basis for calling the reference independent of the MPI update, not a claim that every line of validation code is unrelated to the implementation.

The benchmark script also rejects a timed run if its reported analytic maximum error exceeds 10^{-10}. This is a useful guard against silently graphing an obviously incorrect field, although it is less detailed than the small-grid cell-wise tests. The strong and weak CSV errors are near floating-point roundoff and do not grow with the dramatic runtime increase. In other words, the available evidence points to a performance issue, not a numerical failure in the measured runs.

V EXPERIMENTAL METHOD

The provided Vanda-labelled CSVs [4] identify host `CN-017` and job `1508846.stdct-mgmt-02`. No CPU model, compiler/MPI versions, node allocation details, or raw scheduler output are included in the package. Each configuration has three timed repetitions, and the CSV reports their median. The benchmark script defaults to one warmup run, although the CSV does not record the warmup setting used for this job. The timed region is the iteration loop, including per-step communication and computation but excluding allocation, initialization, analytic checks, field gathering, file output, and process launch. The MPI time reported by the program is the maximum elapsed time across ranks. We use the packaged Vanda-labelled measurements for every performance claim below; separate local CSV files are not used for the report's runtime figures. In the supplied plots, the horizontal axis labelled "allocated CPU cores" is the recorded product RT; it does not independently verify the scheduler allocation or physical core binding.

Strong scaling fixes $(n_x, n_y, K) = (1024, 1024, 400)$ and varies R and T. With serial median t_s, speedup and efficiency are

$$S(R,T) = \frac{t_s}{t_{R,T}}, \qquad E_{\text{strong}}(R,T) = \frac{S(R,T)}{RT}. \tag{7}$$

Weak scaling fixes approximately the number of grid cells per core and $K = 400$: the one-core base is 512^2, the two-core grid is 724^2, and the four-core grid is 1024^2. The weak-scaling efficiency is

$$E_{\text{weak}}(R,T) = \frac{t_{s,512^2}}{t_{R,T,\text{scaled}}}. \tag{8}$$

The 724^2 grid has 262,088 cells per core at two cores, versus 262,144 at one and four cores; the small difference is due to rounding. Weak-scaling efficiency is a runtime ratio, not a strong-scaling speedup. All measured parallel configurations use `static` scheduling; no measured claim about `dynamic` scheduling is made.

Table 2 summarizes what changes between the two experiments. Comparing runs at the same core count within a scaling experiment is especially informative because it isolates the rank/thread mix while retaining the same global grid and iteration count. By contrast, a strong-scaling time and a weak-scaling time should not be directly interpreted as an algorithmic speedup against one another: their grid sizes or baselines differ.

Table 2: Experimental definitions reconstructed from the supplied Vanda CSVs and benchmark script.

Item	Strong scaling	Weak scaling
Global grid	1024×1024 for every run	512^2, 724^2, or 1024^2 according to core count
Iterations	400	400
Serial baseline	1024^2, 400 steps; 0.252533 s	512^2, 400 steps; 0.063858 s
Parallel configurations	$R \in \{1,2,4\}$, $T \in \{1,2\}$ where $RT \le 4$	The same rank/thread configurations
Comparison metric	$t_s/t_{R,T}$ and $(t_s/t_{R,T})/(RT)$	$t_{s,512^2}/t_{R,T,\text{scaled}}$

The CSV files store the three individual elapsed times, their median, the analytic error, hostname, and job identifier. Their speedup and efficiency columns agree with Equations (7)–(8) when recalculated from the recorded medians. The recorded job identifier is the same in both files, so these are two experiments within one identified job rather than independent runs on different job allocations. The package does not include a second job with which to assess between-job variability. Only up to four cores appear in the data, even though an example batch script requests eight CPUs; the report does not extrapolate to an unmeasured eight-core result.

VI RESULTS AND DISCUSSION

A STRONG SCALING

Table 3 lists all supplied strong-scaling medians. At one rank and one thread, the parallel executable takes 0.2933 s in blocking mode and 0.2604 s in overlap mode, compared with the serial 0.2525 s. With two threads but one MPI rank, the medians are 0.2808 and 0.2889 s. Thus extra threads did not improve this particular single-rank measurement. The striking change occurs when two MPI ranks are introduced: both halo strategies take about 10.41 s with one thread per rank. At the same total of two cores, one rank with two threads remains below 0.29 s. The configuration and timing are associated, but the CSV alone cannot identify the underlying cause.

Table 3: Vanda-labelled strong-scaling results, 1024^2 grid, 400 steps. Times are medians of three iteration-loop runs; the serial median is 0.252533 s.

R	T	Cores	Mode	Time (s)	Speedup	Efficiency
1	1	1	Serial	0.2525	1.0000	1.0000
1	1	1	Blocking	0.2933	0.8610	0.8610
1	1	1	Overlap	0.2604	0.9698	0.9698
1	2	2	Blocking	0.2808	0.8994	0.4497
1	2	2	Overlap	0.2889	0.8741	0.4370
2	1	2	Blocking	10.4088	0.0243	0.0121
2	1	2	Overlap	10.4110	0.0243	0.0121
2	2	4	Blocking	10.4107	0.0243	0.0061
2	2	4	Overlap	10.3831	0.0243	0.0061
4	1	4	Blocking	22.4359	0.0113	0.0028
4	1	4	Overlap	11.2870	0.0224	0.0056

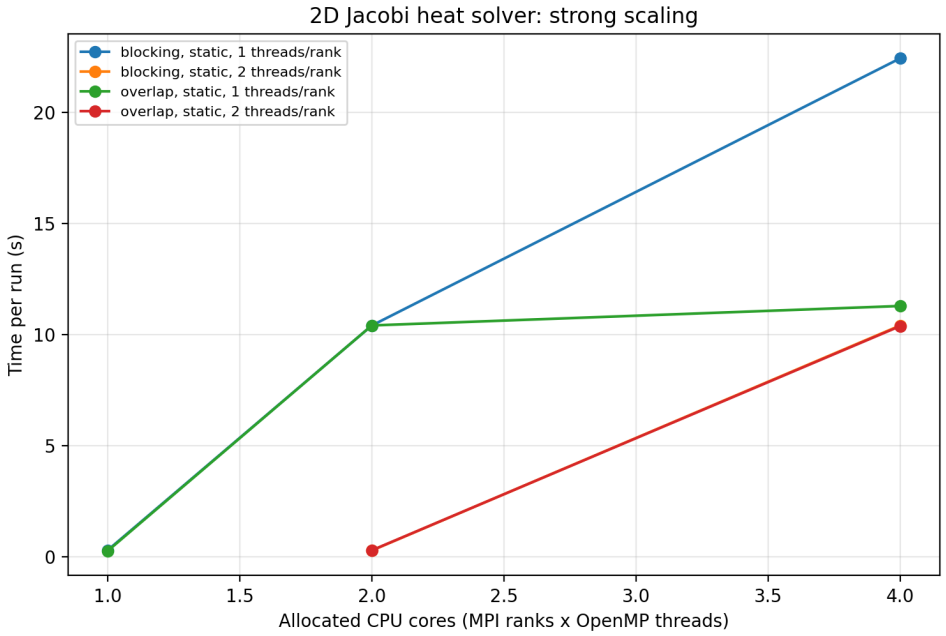


Figure 1: Strong-scaling iteration-loop runtime from the supplied Vanda-labelled CSV. Separate curves distinguish halo mode and threads per rank; points with the same core count can represent different MPI/OpenMP configurations.

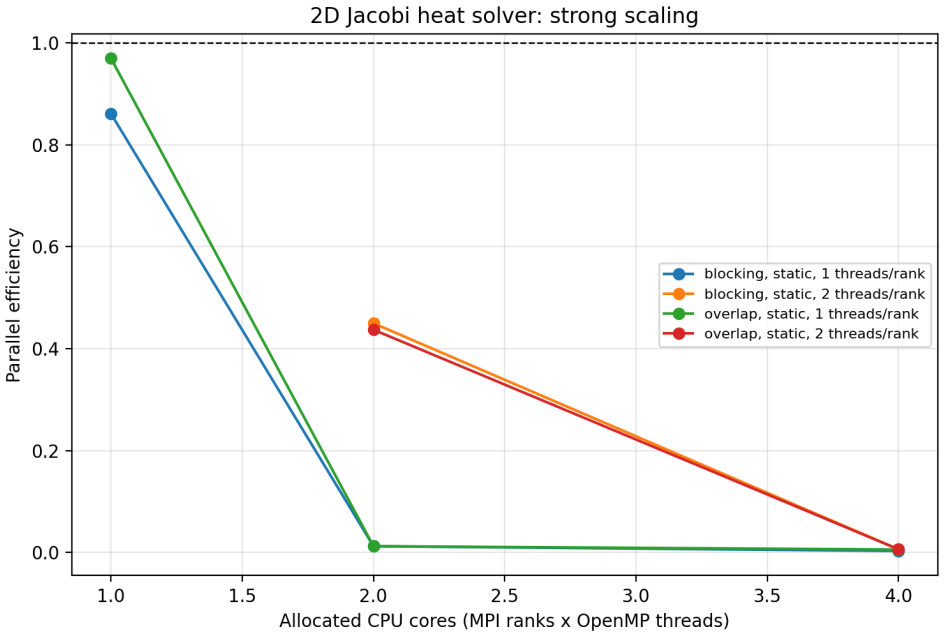


Figure 2: Strong-scaling efficiency computed with Equation (7). The dashed horizontal line indicates ideal efficiency.

At four ranks with one thread each, overlap mode's 11.2870 s median is about half the blocking mode's 22.4359 s median. This is an observed benefit for that configuration, not a general proof that nonblocking communication always helps: with two ranks the two modes are nearly equal, and neither produces useful speedup relative to serial. The supplied record contains neither a communication/compute timing breakdown nor enough environment information to isolate the reason for the roughly 10 s transition. In particular, halo traffic is a plausible contributor but is not established as the sole cause.

The magnitude is easier to see per iteration. Dividing by 400 gives about 0.631 ms for the serial baseline, 0.702 ms for one rank/two threads in blocking mode, 26.0 ms for two ranks/one thread, and 56.1 ms for four ranks/one thread in blocking mode. The two-rank time is approximately 41.2 times the serial time despite a smaller local grid on each rank. Four-rank blocking is approximately 88.8 times the serial time; four-rank overlap is approximately 44.7 times. These ratios describe the measured iteration loop, not end-to-end application runtime.

The fixed-core comparisons sharpen the observation. At two cores, changing from 1×2 to 2×1 raises the blocking median from 0.2808 to 10.4088 s. At four cores, 2×2 and 4×1 both use MPI, but the blocking median rises from 10.4107 to 22.4359 s with more ranks; in overlap mode the corresponding values are 10.3831 and 11.2870 s. The thread count does not substantially change the roughly 10.4-s medians at two ranks, which is consistent with an overhead outside the parallel row calculation dominating those runs. It does not by itself identify that overhead. Differences in rank placement, MPI progress, and process binding remain possible and were not measured in the CSV.

At four ranks, overlap saves $22.4359 - 11.2870 = 11.1489$ s relative to blocking, a reduction of about 49.7% in measured time. At two ranks, however, the modes differ by only about 0.0022 s for one thread per rank. These observations discourage a global statement such as "overlap is twice as fast." They suggest that the effect of the communication schedule depends on rank count and runtime conditions. The dataset supports the configuration-specific comparison, not an explanation of its mechanism.

B WEAK SCALING

Table 4 shows the weak-scaling medians. The serial base is 0.063858 s for 512^2 cells. With one MPI rank and two threads, the scaled 724^2 problem takes 0.1456–0.1470 s, yielding about 0.44 weak-scaling efficiency. With two ranks and one thread each, the same 724^2 problem takes about 10.4 s and efficiency falls to about 0.006. Four ranks show the same large blocking-versus-overlap separation seen in strong scaling. Since the number of cells per core is nearly constant, the increase in time is not explained merely by each core receiving a larger grid; the measurements do not identify which additional costs dominate.

The 1×1 blocking weak-scaling time is slightly below the serial baseline, giving an efficiency just above one. This small difference between separate executables and three-run medians is not evidence of superlinear scaling. For 1×2, the enlarged grid contains almost exactly twice as many cells as the base, but blocking time is about 2.28 times the serial base; efficiency is 0.4386. At 2×1, time is about 162.5 times the base and efficiency drops to 0.00615. At 4×1, blocking takes about 351.5 times the base and overlap about 176.6 times. These weak-scaling ratios should not be divided by core count again; Equation (8) already expresses fixed-work-per-core efficiency.

Both strong and weak experiments exhibit a near-constant two-rank runtime of roughly 10.4 s despite different global grids: 1024^2 in strong scaling and 724^2 in the two-core weak case. Such similarity is a diagnostic clue because pure stencil arithmetic would change with cell count. It is not a diagnosis: the recorded time combines computation, synchronization, MPI operations, and any runtime effects inside the measured loop. Only a new experiment that separates those components can establish whether a fixed per-iteration cost, process placement, or another factor explains the pattern.

Table 4: Vanda-labelled weak-scaling results, 400 steps. Base serial median: 0.063858 s on 512^2. "Efficiency" is Equation (8).

R	T	Grid	Mode	Time (s)	Efficiency
1	1	512^2	Serial	0.06386	1.0000
1	1	512^2	Blocking	0.06213	1.0278
1	1	512^2	Overlap	0.06422	0.9944
1	2	724^2	Blocking	0.14559	0.4386
1	2	724^2	Overlap	0.14700	0.4344
2	1	724^2	Blocking	10.38026	0.0062
2	1	724^2	Overlap	10.41399	0.0061
2	2	1024^2	Blocking	10.39841	0.0061
2	2	1024^2	Overlap	10.39908	0.0061
4	1	1024^2	Blocking	22.44695	0.0028
4	1	1024^2	Overlap	11.27999	0.0057

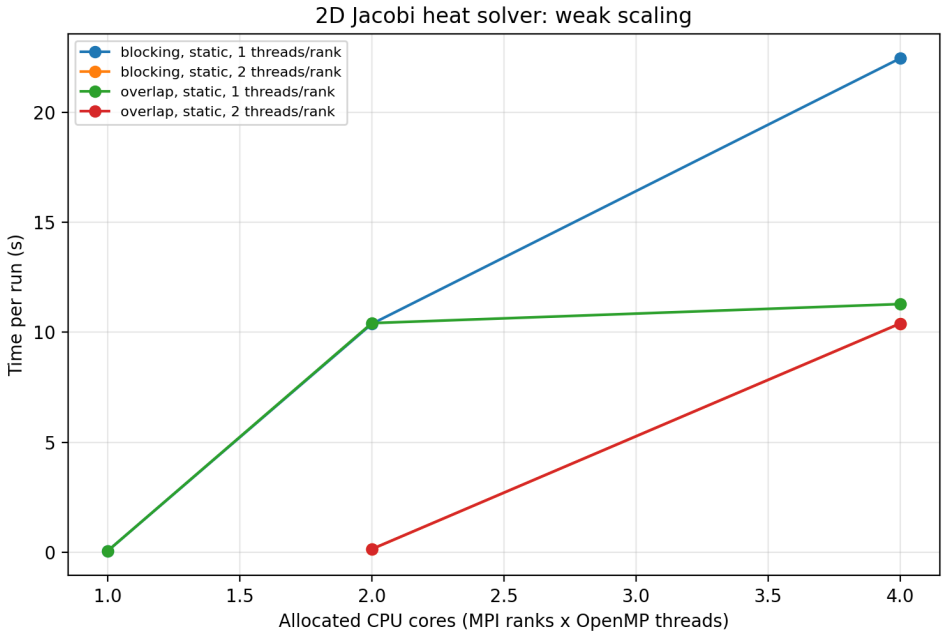


Figure 3: Weak-scaling iteration-loop runtime from the supplied Vanda-labelled CSV. The global grid grows with core count to keep cells per core nearly fixed.

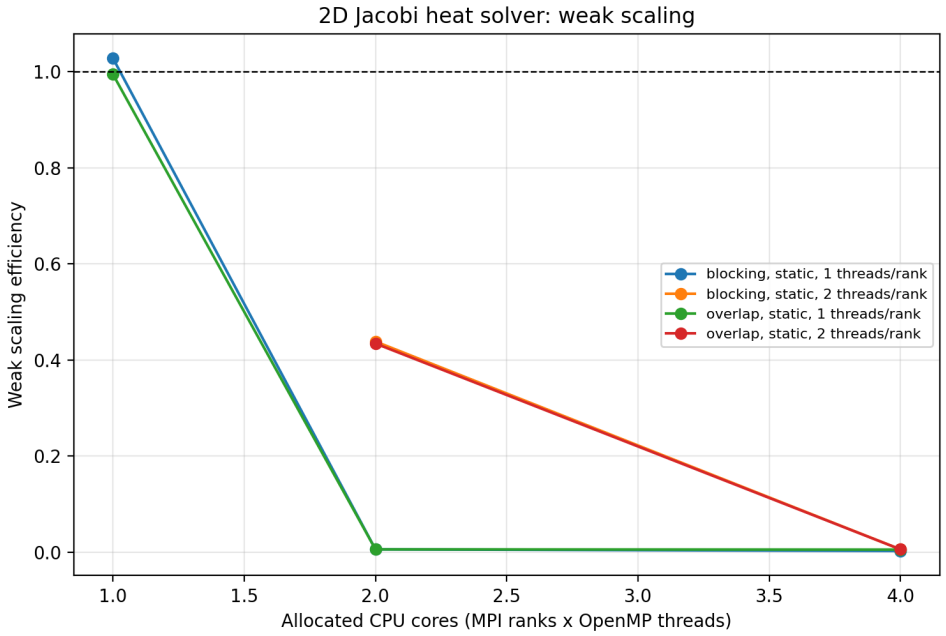


Figure 4: Weak-scaling efficiency computed with Equation (8). The dashed horizontal line indicates ideal efficiency.

C LIMITS OF INTERPRETATION

The very short serial runs, only three timed repetitions per configuration, and the missing machine and placement details limit causal interpretation. The two-rank medians near 10.4 s appear in both scaling experiments despite different grid sizes, which is worth investigating, but the CSV contains no per-step timings, binding information, or raw job log. The code prints `mpi_wait_seconds` and `compute_seconds`, but the benchmark CSV does not retain these fields. Furthermore, `mpi_wait_seconds` has different scopes in blocking and overlap modes, so even those values would need careful interpretation. A targeted rerun recording process placement and timing components would distinguish implementation, MPI environment, and hardware effects more convincingly. The present report does not substitute such a diagnosis with speculation.

The individual repeat values provide a limited stability check. For the strong serial baseline, the three times span 0.24698–0.30387 s, a range equal to about 22.5% of the median. The one-rank blocking parallel times span 0.28843–0.42113 s. These short baselines are sensitive to run-to-run variation, making small differences among one-rank modes inconclusive. In contrast, the strong two-rank blocking runs span 10.38753–10.41026 s, only about 0.22% of their median; four-rank blocking spans 22.41024–22.43994 s. Thus the severe multi-rank slowdown is repeatable within this single recorded job. Three repeats in one job do not establish reproducibility across allocations, nodes, builds, or MPI configurations.

The provided data cannot distinguish among several mechanisms. Every step with more than one rank requires communication, but the measured MPI time includes waiting, possible transport/progress costs, and synchronization with other ranks. The timing pattern could also be affected by process placement or affinity, which the CSV does not record. For the four-rank blocking/overlap gap, the different order of MPI calls is a plausible explanation, but the absence of retained timing components prevents a causal attribution. Even a recorded `mpi_wait_seconds` would not alone measure total MPI overhead: in overlap mode it starts immediately before `MPI_Waitall`, excluding request-posting time and communication that progresses during computation.

A discriminating follow-up would repeat the same fixed-size grid with explicit rank placement and core binding, capture the full `RESULT` lines including wait and compute fields, and run multiple independent allocations. A larger K or grid could test whether the short serial baseline magnifies fixed overhead, while a halo-only microbenchmark could isolate exchange latency. These are proposed checks, not experiments represented in the current tables. Until they are performed, the scientifically supported conclusion is a large measured loss of efficiency in this job and an observed configuration-specific benefit of overlap at four ranks.

VII CONCLUSION

The serial and hybrid codes implement the same fixed-step Jacobi recurrence. The supplied regression record and analytic-error columns support numerical correctness for the tested cases. In the supplied Vanda-labelled measurements, however, the hybrid program does not scale effectively: adding MPI ranks produces a large runtime increase, while nonblocking exchange improves the four-rank result relative to blocking exchange without restoring a speedup over serial. The principal performance limitation cannot be uniquely identified from the packaged records. This is a measured result and a limitation of the evidence, rather than a claim that MPI+OpenMP is inherently ineffective for Jacobi iteration.

The design comparison still yields useful lessons. The same numerical update can be organized with different communication schedules; their benefit is conditional rather than automatic. Holding total cores fixed reveals a sharp difference between thread-only and multi-rank configurations, while weak scaling shows that keeping work per core approximately constant does not preserve runtime in the supplied environment. The exact eigenmode reference and cell-wise regression tests support correctness independently of these performance outcomes. A repeat experiment with recorded MPI timing components and placement information is needed before assigning the slowdown to a particular layer of the implementation or machine.

A REPRODUCTION AND EVIDENCE INDEX

The source package contains `src/serial.cpp`, `src/parallel.cpp`, and shared numerical routines in `src/common.hpp`. The `Makefile` builds `build/heat_serial` and `build/heat_parallel`. The project README gives command-line options and describes the compilation prerequisites. The commands below illustrate how to rebuild and validate within a suitable compute allocation where a C++ compiler, an MPI implementation, OpenMP, and Python are available:

```
make
python3 scripts/validate.py
```

The benchmark script can generate a configuration set matching the recorded grid sizes and step counts with commands of the following form. They are reproduction instructions, not a claim that the missing original scheduler command line was recovered:

```
python3 scripts/benchmark.py --scaling strong \
  --nx 1024 --ny 1024 --steps 400 \
  --ranks 1,2,4 --threads 1,2 --max-cores 4 \
  --repeats 3 --warmup 1 --output results/strong_scaling.csv

python3 scripts/benchmark.py --scaling weak \
  --nx 512 --ny 512 --steps 400 \
  --ranks 1,2,4 --threads 1,2 --max-cores 4 \
  --repeats 3 --warmup 1 --output results/weak_scaling.csv
```

The included PBS and Slurm files are examples that require adjustment to the actual cluster environment. In particular, the supplied example batch scripts specify 300 weak-scaling steps, whereas the provided Vanda weak-scaling CSV records 400 steps. Reproducing the reported weak-scaling comparison requires using the CSV's 400-step setting. The original job's exact launch command, allocation, module versions, and scheduler output are not part of this ZIP; this report does not infer them from an example script.

Table 5: Evidence in the supplied Project 3 ZIP and its role in this report.

Package item	Evidence or purpose
`src/common.hpp`	Recurrence, initialization, analytic metrics,
`src/serial.cpp`	decomposition, halo exchange, and timing scope.
`src/parallel.cpp`	
`scripts/validate.py`	Regression cases, tolerances, and reported field-wise
`results/validation_log.txt`	passes; log environment is not recorded.
`scripts/benchmark.py`	Configuration loops, three-run medians, default warmup, and CSV metric definitions.
`results_vanda/vanda_strong_scaling.csv`	Strong-scaling raw repeats, medians, errors, host, and job ID.
`results_vanda/vanda_weak_scaling.csv`	Weak-scaling raw repeats, medians, grid sizes, errors, host, and job ID.
`results_vanda/*.png`	The four runtime/efficiency figures reproduced in this report.

The two CSVs contain no eight-core configuration and no dynamic-scheduling performance series. The package also contains older local CSVs in `results/`; they are not combined with the Vanda figures or used to calculate the tables in this report. Some earlier README and handoff statements say that cluster results were not yet included, whereas the later `results_vanda/` files are present in the supplied ZIP. We use the latter as the performance dataset without treating those earlier statements as a record of the subsequent job. Retaining the datasets' separation is essential because comparisons across machines, builds, or runtime environments would not be controlled by a shared serial baseline.

REFERENCES

[1] R. Barrett et al., *Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods*, 2nd ed. Philadelphia, PA: SIAM, 1994. `https://netlib.org/templates/templates.html`

[2] MPI Forum, *MPI: A Message-Passing Interface Standard, Version 4.1*, 2023. `https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf`

[3] OpenMP Architecture Review Board, *OpenMP Application Programming Interface, Version 5.2*, 2021. `https://www.openmp.org/spec-html/5.2/openmp.html`

[4] Fan Weiqi, Liu Peihan, and Chen Yiru, *Project 3: Hybrid MPI+OpenMP 2D Jacobi Heat Solver*, source code, validation log, and Vanda-labelled benchmark CSVs supplied with the project ZIP, 2026.

AI-USE DISCLOSURE

The supplied README states that AI assistance helped draft or structure parts of the code package and its documentation. OpenAI Codex was also used to review the supplied files, check the CSV calculations, organize the analysis, and draft this report. All reported performance values and figures were taken from, or calculated from, the supplied Vanda-labelled CSVs and images, no new measurement values were invented or altered while preparing this report. The assignment brief and package documentation supplied by the team were used as project sources.